



Git

Git и GitHub

Гид для начинающих

Commit · Push · Pull · Pull Request



Объясняем на пальцах.
Ну, почти. На бабушках.

Что такое Git и GitHub?

Git — это система контроля версий. Она запоминает каждое изменение в ваших файлах и позволяет в любой момент вернуться назад. Представьте себе «машину времени» для документов и кода.

GitHub — это онлайн-платформа, где хранятся проекты. Облачное хранилище с суперсилами: полная история изменений, совместная работа и возможность предлагать правки другим людям.

А теперь — на примере бабушки

Представьте, что вы рисуете картинку. Рядом сидит бабушка, которая обожает хранить всё на свете. У неё дома — бесконечный шкаф, куда влезут все ваши рисунки. А ещё у неё живёт ваш друг Вася, который тоже любит рисовать.

В этой аналогии:

- **Вы** = разработчик (работаете за своим компьютером)
- **Бабушка** = GitHub (надёжное хранилище в облаке)
- **Рисунок** = ваш проект (код, документы, файлы)
- **Вася** = ваш коллега

4 главных понятия

- **Commit** — сфотографировать рисунок («сохраняшка»)
- **Push** — отправить фотографии бабушке
- **Pull** — забрать у бабушки то, что нарисовал Вася
- **Pull Request** — вежливо попросить: «Можно дорисую?»

COMMIT — «Сохраняшка»

Бабушкина версия: Вы рисуете картинку. Нарисовали солнышко — и фотографируете рисунок. Подписываете: «Добавил солнышко». Нарисовали домик — снова фотографируете: «Добавил домик». Каждая такая фотография — это коммит. Можно всегда вернуться и посмотреть, как рисунок выглядел на любом этапе.

Что это по-настоящему: Вы написали код, исправили баг или отредактировали документ. Команда **git commit** сохраняет снимок всех изменений с вашим комментарием. Теперь в истории есть точка, к которой можно откатиться.

Зачем: Не бояться экспериментировать — всегда можно вернуться. Видеть полную историю: кто, когда и что менял. Каждый коммит — это точка восстановления для вашего проекта.

PUSH — «Отправить бабушке»

Бабушкина версия: Фотографии рисунка пока только у вас. А бабушка волнуется! Push — это когда вы берёте все фотографии и отправляете их бабушке по почте. Теперь копии в безопасности у неё в шкафу. Даже если у вас дома потоп — рисунки не пропадут.

Что это по-настоящему: Команда **git push** отправляет ваши локальные коммиты на GitHub. После этого коллеги могут увидеть ваши изменения, а у проекта появляется надёжная резервная копия.

Зачем: Резервная копия в облаке — компьютер может сломаться, а код на GitHub останется. Коллеги получают доступ к вашей работе. Это мост между вашим компьютером и всей командой.

PULL — «Забрать у бабушки новенькое»

Бабушкина версия: Пока вы спали, ваш друг Вася тоже рисовал и отправил бабушке свои фотографии. Он добавил на картинку кота! Вы делаете pull — и хоп! — у вас на столе обновлённый рисунок уже с Васиным котом. Pull — это «бабушка, дай всё новенькое!»

Что это по-настоящему: Команда `git pull` скачивает свежие изменения с GitHub на ваш компьютер. Это привычка: пришли на работу — сделали pull, чтобы получить всё, что коллеги добавили за ночь.

Зачем: Быть в курсе изменений команды. Не работать с устаревшей версией. Избежать конфликтов — когда два человека правят одно и то же место, не зная друг о друге.

PULL REQUEST — «Можно я добавлю?»

Бабушкина версия: Вы хотите дорисовать дракона на **чужой** картинке. Но нельзя просто так взять и нарисовать — вдруг хозяину не понравится! Поэтому вы рисуете дракона на отдельном листочке и говорите: «Смотри, я тут дракона нарисовал. Хочешь — добавь к себе?» Хозяин смотрит, говорит «класс!» — и добавляет. Или просит: «Перерисуй хвост» — и вы исправляете.

Что это по-настоящему: Pull Request (PR) — это запрос на включение ваших изменений в основной проект. Вы создаёте PR на GitHub, коллеги смотрят код, оставляют комментарии, предлагают правки. Когда все довольны — PR «мёрджат» (принимают).

Зачем: Контроль качества — четыре глаза лучше двух. Обсуждение решений прямо в контексте кода. Защита основной версии проекта от случайных ошибок.

Как это работает вместе

- 1 Рисуете**
Работаете с файлами: код, текст, дизайн...
- 2 Фотографируете**
Commit — сохраняете снимок с подписью
- 3 Отправляете бабушке**
Push — загружаете на GitHub
- 4 Забираете Васиные рисунки**
Pull — скачиваете обновления коллег
- 5 «Можно дорисую?»**
Pull Request — предлагаете изменения на проверку

Шпаргалка по командам

<code>git init</code>	Создать новый репозиторий
<code>git clone <url></code>	Скопировать чужой проект к себе
<code>git add .</code>	Подготовить файлы к коммиту
<code>git commit -m "текст"</code>	Сохранить снимок изменений
<code>git push</code>	Отправить коммиты на сервер
<code>git pull</code>	Получить обновления с сервера
<code>git status</code>	Посмотреть, что изменилось
<code>git log</code>	Посмотреть историю коммитов

Золотое правило

Коммитьте часто, пушьте регулярно, пуллите перед началом работы. Каждый коммит — одно логичное изменение с понятным описанием. «Исправил баг авторизации» — хорошо. «Разные правки» — плохо. Бабушка любит порядок!

Вы **работаете** с файлами на своём компьютере.

Делаете **commit** — фотографируете текущее состояние: «Вот что я сделал».

Делаете **push** — отправляете всё бабушке (на GitHub), чтобы было в безопасности.

Делаете **pull** — забираете у бабушки всё, что Вася (коллега) туда добавил.

Создаёте **pull request** — вежливо предлагаете свои изменения: «Посмотри, может добавишь?»

Кто этим пользуется?

- **Программисты** — хранят код, работают в команде, ревьюят изменения друг друга.
 - **Дизайнеры** — версионировать макеты, собирают фидбек через PR.
 - **Писатели и редакторы** — ведут историю правок статей и книг.
 - **Учёные** — хранят данные исследований, делятся ими с коллегами.
 - **Кто угодно** — любой, кто работает с файлами и хочет порядок и историю.
-

Вот и всё. Четыре понятия, одна бабушка, и вы уже понимаете Git лучше, чем 80% людей, которые только собираются начать.